Starting and stopping the service without losing a request

Shutdown is the delicate half: it must stop workers blocked on an empty queue without abandoning work already in flight

1. Start

Allocate the shards, spawn one worker thread per shard, and hand each worker exclusive ownership of its store. Nothing is shared, so nothing needs locking.

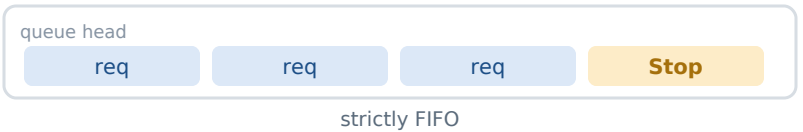
2. Run

A request is moved into the queue rather than copied, carrying its completion handle with it, so the producer and the worker never both hold a usable reference to it.

3. Stop

Push one Stop sentinel onto each shard's queue, then join the workers, so shutdown returns only once every shard has actually stopped.

Why an in-band sentinel needs no drain API



The worker observes Stop only after the three requests ahead of it have run and completed, so nothing is abandoned.

Tied to object destruction

The C++ destructor calls shutdown() and Rust's Drop does the same, so a service that goes out of scope cannot leave detached threads behind. A running flag makes the call idempotent on both sides.

The one piece of shared mutable state

An atomic activity counter, held by reference in C++ and as a cloned Arc<AtomicU64> in Rust, incremented with relaxed ordering because nothing branches on its value. Write-only bookkeeping.

No separate closed flag, timeout, or drain API is required, and no completion handle is left unfulfilled.